Cairo University
Faculty of Engineering
Electronics & Communications Dept.

ANN COURSE PROJECT

LinearMind Platform

Technical Report

Prepared by:

Abdallah Essam Mohamed Mahmoud	ID: 9220479
Zeyad Antar Othman Abu-Zaid	ID: 9220331
Abdelrahman Hatem Nasr Youssef	ID: 9220461

Submitted to:

Prof. Dr. Mohsen Rashwan

Dr. Mohamed Abdelghani

LinearMind App **GitHub Repository** **Demo Video**

May 2026

Contents

1 Introduction **4**

2 Aim of the Project **4**

3 Concept Overview: Linear Regression **5**

3.1 Connection to Neural Networks 5

4 Block Diagram of Steps **6**

4.1 Step-by-Step Breakdown 6

5 System Architecture **7**

6 AI Tools Used **7**

7 Key Features **8**

7.1 Interactive Regression Playground 8

7.2 15 Interactive Visualizations 9

7.3 AI Tutor (Gemini-Powered Chatbot) 9

7.4 Quiz System 9

7.5 Gamification 10

7.6 Google Authentication 10

8 Technology Stack **11**

9 Challenges **11**

10 Conclusions **12**

List of Figures

1 Development flow of the LinearMind platform. 6

2 System architecture layers. 7

List of Tables

1 AI tools used in the project. 8
2 All interactive visualizations in LinearMind. 9
3 Technology stack. 11

List of Abbreviations

Abbreviation	Full Form
ANN	Artificial Neural Network
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
FPS	Frames Per Second
GPU	Graphics Processing Unit
JS	JavaScript
LLM	Large Language Model
MAE	Mean Absolute Error
MLP	Multi-Layer Perceptron
MSE	Mean Squared Error
OAuth	Open Authorization
R^2	Coefficient of Determination
RMSE	Root Mean Squared Error
SSR	Server-Side Rendering
TS	TypeScript
UI	User Interface
XP	Experience Points

Abstract

In this project, we wanted to find a way to explain Linear Regression in a way that actually makes sense to someone seeing it for the first time. Instead of writing another textbook-style explanation, we decided to build an interactive website called **LinearMind** where users can play with the math themselves. The platform lets you click on a canvas to place data points, watch gradient descent optimize the line in real time, adjust the learning rate and see what happens, and even chat with an AI tutor powered by Google Gemini if you get stuck. We included 15 different visualizations, a 13-module curriculum with 29 lessons, 83 quiz questions, and a progress system with XP and achievements to keep things engaging. The whole thing is built with Next.js, React, and Firebase. Most importantly, we use the platform to show that a single neuron ($y = \mathbf{w}^T\mathbf{x} + b$) is really just linear regression with extra steps, which we think is a useful insight for anyone starting out with neural networks.

1 Introduction

When we first learned about Linear Regression in class, it seemed like a straightforward algorithm. You fit a line through data points, minimize the error, and you are done. But then we started looking at neural networks and realized that at the core of every single neuron is the exact same equation: $z = \sum w_i x_i + b$. That connection between a "simple" regression line and a neural network felt like something worth exploring.

The problem is, most learning resources explain regression through static formulas and graphs on paper. When we were studying gradient descent, for example, it was hard to picture what was actually happening just by reading the update rule. We kept asking ourselves: what does it look like when the learning rate is too high? How does an outlier actually pull the line? Why does polynomial regression overfit so easily?

We figured the best way to answer those questions was to build something interactive. So we created **LinearMind**, a web platform where you can:

- Place data points on a canvas and see the regression line update instantly
- Run gradient descent step by step and watch the parameters converge
- Try different learning rates, add outliers, switch to polynomial regression
- Chat with an AI tutor (Google Gemini) when something does not click
- Take quizzes to check your understanding

The rest of this report covers the project aim (Section 2), the math behind it (Section 3), how we built it (Sections 4 and 5), the AI tools we used (Section 6), the main features (Section 7), the tech stack (Section 8), the challenges we ran into (Section 9), and what we concluded from the whole experience (Section 10).

2 Aim of the Project

Our goal was to pick one ANN concept and build something that actually helps people understand it. We chose **Linear Regression** because it is where everything in neural networks starts. A neuron computes $y = wx + b$, which is literally the regression equation, so if you understand regression well, the jump to neural networks becomes much easier.

Rather than just making a presentation or a document, we wanted to create something hands-on. So we built **LinearMind**, a website where you can interact with regression concepts directly. It has:

- 15 visualizations you can play with (gradient descent, cost functions, residuals, etc.)
- A playground where you click to place points and the regression line updates live

- An AI chatbot (Gemini) that answers your questions about the topic
- 13 modules with 29 lessons that walk you through everything step by step
- 83 quiz questions so you can test yourself
- XP, streaks, and achievements to keep things fun

The big idea we wanted to get across is that linear regression is not just some old statistics method. It is the basic unit of every neural network.

3 Concept Overview: Linear Regression

At its core, linear regression tries to find the best straight line through a set of data points. Given input x and output y, the model fits a linear equation:

$$\hat{y} = w \cdot x + b \tag{1}$$

Here w is the slope and b is the intercept. To find the best values for w and b, we minimize the Mean Squared Error (MSE), which measures how far off our predictions are:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{2}$$

We use **Gradient Descent** to iteratively update the parameters toward the minimum:

$$w \leftarrow w - \alpha\frac{\partial \text{MSE}}{\partial w} \tag{3}$$

$$b \leftarrow b - \alpha\frac{\partial \text{MSE}}{\partial b} \tag{4}$$

where α is the learning rate.

3.1 Connection to Neural Networks

A single artificial neuron computes:

$$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T\mathbf{x} + b \tag{5}$$

If you strip away the activation function, this is just multivariable linear regression. And the way we train neural networks (backpropagation) is really gradient descent applied one layer at a time using the chain rule. So linear regression is the smallest building block of any neural network.

4 Block Diagram of Steps

Here is how we went about building the platform, from choosing the topic to deploying the final product:

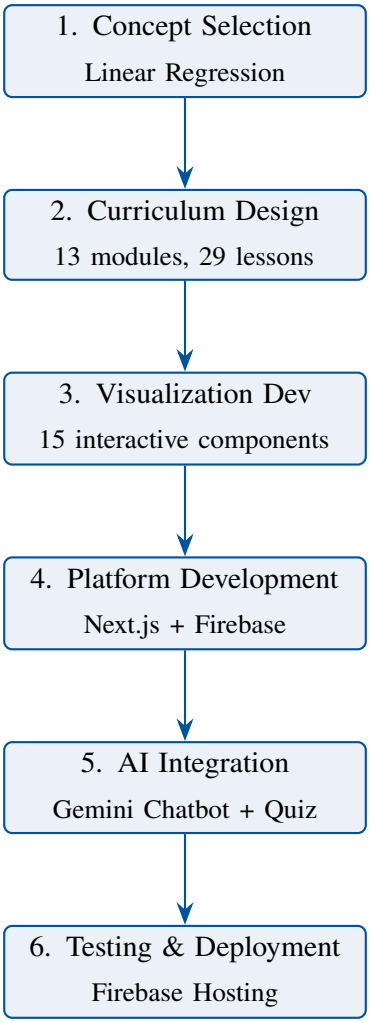

Figure 1. Development flow of the LinearMind platform.

4.1 Step-by-Step Breakdown

1. **Concept Selection:** We went with Linear Regression because it connects directly to how neurons work. It felt like a natural starting point.
2. **Curriculum Design:** We mapped out 13 modules that take you from the basics ("what is a line?") all the way to "a neuron is just linear regression." We tried to make the content tell a story rather than just list formulas.
3. **Visualization Development:** This was the most time-consuming part. We coded 15 interactive components using HTML5 Canvas and React:
   - Slope-intercept explorer with sliders

- Cost function (MSE) visualizer
- 2D and 3D gradient descent animations
- Live training loop with loss charts
- Residual analysis plots
- Bias-variance tradeoff visualization
- Neuron-to-regression bridge visualization

4. **Platform Development:** We used Next.js 16 with React 19 and TypeScript for the frontend, Tailwind CSS for styling, Zustand for managing state, and Firebase for login and saving user progress.
5. **AI Integration:** We hooked up Google Gemini 3 Flash as an in-app tutor that can explain things in 4 different ways depending on who is asking. We also wrote 83 quiz questions at different difficulty levels.
6. **Testing & Deployment:** We deployed everything on Firebase Hosting and used Firestore so users can pick up where they left off on any device.

5 System Architecture

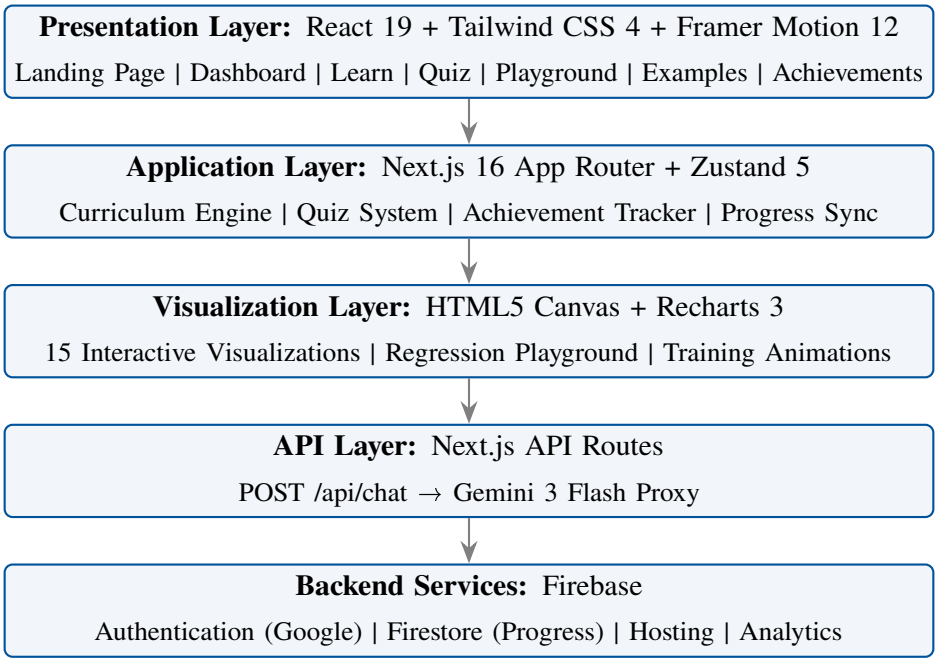

Figure 2. System architecture layers.

6 AI Tools Used

We used several AI tools during development, as encouraged by the project guidelines:

AI Tool	Role	Usage
GitHub Copilot	Code Assistant	Assisted in writing React components, visualization logic, and Firestore integration
Google Gemini 3	AI Tutor (In-App)	Integrated into the platform as an interactive chatbot that explains linear regression concepts

Table 1. AI tools used in the project.

We want to be clear that all the content and code is our own work. The AI tools helped us move faster and debug issues, but we did not copy anything from other sources.

7 Key Features

7.1 Interactive Regression Playground

This is probably the part we are most proud of. It is basically a sandbox where you can play with regression however you want:

- Click to add data points and watch the regression line update in real-time
- Toggle residual lines, confidence bands, and grid overlays
- Manually adjust slope and intercept with sliders and compare to the optimal fit
- Watch gradient descent train live with adjustable learning rate
- Switch between linear and polynomial regression (degree 2–10)
- Use 6 dataset presets (Linear, Noisy, Outlier, Quadratic, Clusters, No Trend)
- Follow 8 guided experiments

7.2 15 Interactive Visualizations

#	Visualization	Description
1	RegressionCanvas	Scatter plot with live regression line
2	SlopeInterceptViz	Dual sliders for slope and intercept
3	CostFunctionViz	MSE cost function parabola
4	GradientDescentViz	2D animated gradient descent
5	GradientDescent3DViz	3D contour plot with ball descent
6	TrainingViz	Training loop with live loss chart
7	ResidualsViz	Residual scatter + distribution
8	FailureCasesViz	Outliers, nonlinear, heteroscedasticity
9	ComparisonViz	Linear vs polynomial vs ridge
10	MultivariateViz	Multi-feature regression
11	FeatureScalingViz	Min-max vs Z-score normalization
12	CorrelationViz	Correlation coefficient explorer
13	BiasVarianceViz	Underfitting $\leftrightarrow$ overfitting
14	MetricsViz	R^2, MSE, MAE, RMSE comparison
15	NeuronBridgeViz	Linear regression $\rightarrow$ neuron bridge

Table 2. All interactive visualizations in LinearMind.

7.3 AI Tutor (Gemini-Powered Chatbot)

We integrated a floating chatbot that you can open from any page. It has 4 explanation modes so it can adapt to the user:

- **Beginner:** Simple language, everyday analogies
- **Engineer:** Technical terminology, mathematical notation
- **Child:** Explains as if talking to a 10-year-old
- **Math:** Full derivations and proofs

It renders math equations properly using KaTeX and we set it up so it only answers questions about linear regression (it politely redirects off-topic questions).

7.4 Quiz System

83 questions across 4 types:

- **Conceptual:** Theory understanding
- **Visual:** Interpret graphs and visualizations
- **Scenario:** Real-world application problems
- **Prediction:** Calculate or predict outputs

There are three difficulty levels (Easy, Normal, Hard). After each question you get immediate feedback with an explanation of why the answer is correct.

7.5 Gamification

- XP system with leveling
- Daily streak tracking
- 10 unlockable achievements (e.g., "Gradient Master," "Math Wizard")
- Progress rings and dashboard statistics

7.6 Google Authentication

We used Firebase Authentication with Google sign-in because most universities, including ours, use Google-based accounts for students. This means any student can log in instantly with their university email without needing to create a separate account or remember another password. It also lets us track each student's progress, quiz scores, and achievements securely across sessions.

8 Technology Stack

Layer	Technology
Framework	Next.js 16.2 (App Router, Turbopack)
Language	TypeScript 5
UI Library	React 19.2
Styling	Tailwind CSS 4
Animations	Framer Motion 12
Charts	Recharts 3.8
State Management	Zustand 5 (persist middleware)
Authentication	Firebase Authentication (Google)
Database	Cloud Firestore
AI Engine	Google Gemini 3 Flash
Math Rendering	KaTeX (remark-math + rehype-katex)
Hosting	Firebase Hosting

Table 3. Technology stack.

9 Challenges

1. **Keeping the canvas smooth:** We have gradient descent animations, 3D plots, and draggable points all running at the same time. Getting this to run at 60 FPS without lag took a lot of optimization with `requestAnimationFrame` and caching expensive calculations.
2. **Making gradient descent look right:** It was surprisingly hard to animate gradient descent in a way that actually teaches something. We had to experiment a lot with step sizes, arrow visuals, and path trails before it felt intuitive.
3. **The 3D loss landscape:** Drawing a 3D contour plot with an animated ball rolling downhill, all on a flat 2D canvas, required us to implement isometric projection from scratch. This was one of the trickier parts.
4. **Polynomial gradient descent:** We wanted the playground to auto-detect when data is curved and switch to polynomial fitting. Implementing gradient descent for polynomials meant dealing with Vandermonde matrices, which was new to us.
5. **Next.js hydration bugs:** Since Next.js renders pages on the server first, our canvas components kept causing mismatches between server and client. We had to write a custom

`useHydrated` hook and use dynamic imports to fix this.

6. **Keeping the chatbot on topic:** The Gemini chatbot would happily answer questions about anything if we let it. Getting it to stay focused on linear regression without sounding robotic took several rounds of prompt tuning.
7. **Touch support:** Most of our visualizations rely on mouse events (click, drag, hover). Making all of that work on phones with touch events was tedious but necessary.
8. **Syncing progress:** Users have progress stored locally (Zustand) and in the cloud (Firestore). Making sure these stay in sync without overwriting each other or losing data was a headache we did not expect.
9. **Gemini API key setup:** Getting the Gemini API key to work properly was not straightforward. We had to deal with API quotas, rate limits, and making sure the key was stored securely in environment variables without exposing it on the client side.
10. **Database design:** Setting up the Firestore database structure took several attempts. We had to figure out how to organize user data (progress, quiz scores, achievements, streaks) in a way that was efficient to read and write without hitting Firestore's limitations.
11. **Firebase integration:** Connecting the app to Firebase was more involved than we expected. Configuring the Firebase project, setting up the correct security rules in Firestore, and making sure everything worked in both development and production environments required a lot of trial and error.
12. **Google authentication:** Implementing Google Sign-In through Firebase Authentication had its own set of issues. We had to configure OAuth consent screens, handle redirect URLs correctly, and manage auth state persistence across page refreshes and different tabs.
13. **Making it work on both mobile and web:** The platform needed to work well on desktop browsers and mobile phones. This meant dealing with different screen sizes, responsive layouts, and making sure all the interactive canvases and controls were usable on small screens without breaking the experience.

10 Conclusions

Building this project gave us a much deeper understanding of linear regression than we had before. When you have to code gradient descent from scratch and animate every step, you cannot hide behind memorized formulas anymore.

Here is what we took away from the experience:

1. A neuron really is just regression. Once we built the NeuronBridgeViz component and saw $y = \mathbf{w}^T\mathbf{x} + b$ side by side with a neuron diagram, the connection became obvious.
2. Backpropagation is just gradient descent repeated across layers. We already knew this

from lectures, but watching it happen in our training visualization made it click in a different way.

3. Interactivity matters. Being able to drag a data point and instantly see the line shift, or crank up the learning rate and watch gradient descent overshoot, teaches more than reading about it.
4. Having an AI tutor that adapts to your level is genuinely useful. The Gemini chatbot surprised us with how well it could simplify complex topics when set to "Child" mode.

In total, the platform includes 13 modules with 29 lessons, 15 interactive visualizations, 83 quiz questions, 8 code examples in Python/JavaScript/TypeScript, a full playground with live gradient descent, an AI chatbot, and a gamified progress system.

Everything is deployed and accessible online through Firebase Hosting.